



Bagisto Project - Final Report

Nadin Alaskar 222410587

Layan Alhabib 222410739

Norah Alfayez 222410215

Fay Alsubaie 222410396

College of Computer & Information Science

Secure Software Development CYS402

Dr. Zainab Masood

Table of Contents

Phase 1

1. Overview of the Software Domain.....	5
Introduction.....	5
GitHub Repository.....	5
Industry/Domain.....	5
2. Identification of Significant Security Risks.....	6
2.1 Main Security Risks.....	6
2.2 Origins of These Risks.....	7
3. Data Protection.....	7
3.1 Data Types.....	7
3.2 Existing Protection Measures.....	7
3.3 Identified Weaknesses.....	7
4. Consequences of Security Failures.....	8
4.1 Immediate Consequences.....	8
4.2 Long-term Consequences.....	8
4.3 Impact on Users.....	8
4.4 Impact on Company and Stakeholders.....	8

Phase 2

1. Product Assets.....	14
1.2 Asset Prioritization.....	15
1.3 Decision Process.....	16
1.4 Future Changes.....	16
2. Malicious Actors & Threats.....	17
2.1 Types of Attackers.....	17
2.2 High-Level Threats (STRIDE).....	17
2.3 CIA Mapping.....	19
2.4 Available Information to Attackers.....	20
2.5 Exploitation Methods.....	20
3. Software Requirements (SQUARE-Lite Section).....	22
3.1 Agree on Definitions.....	22
3.2 Identify Security Goals.....	23
3.3 Elicit Security Requirements.....	23
3.4 Prioritize Requirements.....	29
4. Team Contribution Table.....	33

Phase 3

Architecture Overview.....	38
System Architecture Overview.....	38
Subsystems Description.....	39
Security Specific Subsystems.....	41

	2
Security Risks Analysis.....	41
Impact of Subsystem Compromise.....	43
Subsystems Susceptibility to Vulnerabilities.....	43
Security Architecture Evaluation.....	44
Threat Model.....	44
Overview.....	44
Machine Boundaries.....	46
Trust Boundaries.....	47
Threat Identification.....	47
Threat Mitigation.....	48
Risk Concentration.....	48
Asset to Threat Model Mapping.....	49
Overview.....	49
Authentication Data.....	49
Payment Data.....	49
Customer Personal Data.....	50
Order Records.....	50
Inventory Data.....	50
Cross-Asset Risk Analysis.....	51
Security Implications.....	51
Detailed Design.....	51
Overview.....	51
User Authentication Design.....	52
Order Processing Design.....	53
Product Management Design.....	54
Key Relationships Summary.....	54
Design Evaluation.....	54
Design Patterns and Security Principles.....	55
Team Contribution.....	56
Phase 4	
Introduction.....	59
GitHub Links.....	59
Manual Code Review.....	59
Admin Login Function.....	59
Customer Login Use Case.....	62
Cart Merge Function.....	65
PayPal IPN Processing Function.....	68
Automatic Static Analysis Using SonarCloud.....	71
Authentication Issue (Nadin).....	74
Regex Denial of Service Hotspot (Layan).....	75

	3
Open Redirect Issue (Norah).....	76
Permissive CORS Policy (Fay).....	77
Team Contribution.....	78
Conclusion.....	78



Bagisto Project - Phase 1

Nadin Alaskar 222410587

Layan Alhabib 222410739

Norah Alfayez 222410215

Fay Alsubaie 222410396

College of Computer & Information Science

Secure Software Development CYS402

Dr. Zainab Masood

1. Overview of the Software Domain

Introduction

Bagisto is an open-source eCommerce platform. It is built on Laravel and Vue.js. Bagisto gives the businesses and the developers a solution to create, manage, and scale their online stores. This platform provides a lot of features like management for the products and the inventories, multi-channel selling, localization, and customization of the store interfaces. All of these features make Bagisto a great solution for a lot of eCommerce applications.

GitHub Repository

The code of Bagisto is uploaded to this GitHub repository: <https://github.com/bagisto/bagisto>

Industry/Domain

Bagisto works on the eCommerce/digital retail domain. In this domain, the applications are not only websites, they are systems that process transactions and the data of the customers. In practice, Bagisto is the center of many business workflows, it handles online store browsing, registration and login of the users, shopping carts and checkout, handling orders and refunds, updating inventories, integrating with shipping services and payment gateways, and administrative reporting. These workflows directly include money and sensitive information, so the eCommerce domain has a higher security risk profile than the usual web applications.

The security in eCommerce platforms like Bagisto should have a lot of attention because they handle and store the following:

- **Customer personal data:** This includes the names, the emails, the phone numbers, the addresses, and the order history.
- **Authentication data:** This includes the passwords, the sessions, the password resets, the access tokens.
- **Order and financial records:** This includes the invoices, the refunds, the tax records, and the transaction references.
- **Business data:** This includes the product categories, the pricing rules, and the discount codes.
- **Secrets Keys for the Integrations:** This includes the API keys for payment gateways, the shipping providers, the email/SMS services, and the analytics tools.

Any security failure could cause a huge financial loss, data leaks, and business failures like taking the stores down. These risks are huge because the eCommerce applications are provided to the users and get targeted a lot by the attackers.

The security in the domain of eCommerce platforms is affected by common types of attacks, such as:

- **Account takeover (ATO):** In this attack, the attackers try to log in using leaked or stolen usernames and passwords, weak passwords.
- **Payment and checkout abuse:** This attack includes changing the prices or the discounts, making a lot of payment requests, or skipping the validation process during checkout.
- **Web vulnerabilities:** This attack depends on issues like SQL injection, cross-site scripting (XSS), CSRF attacks like placing orders.
- **Privilege escalation:** This attack targets getting admin access because of misconfigured roles, permissions, or insecure backend endpoints.
- **Data scraping and leakage:** Automatically collecting customer or product data without permission.
- **Supply-chain risks:** Security problems introduced by third-party plugins, libraries, or external integrations.

2. Identification of Significant Security Risks

2.1 Main Security Risks

Bagisto is an open source e-commerce platform that manages sensitive customer data, transactions, and administrative functions, which exposes it to multiple security risks affecting confidentiality, integrity, and availability. One significant risk is the compromise of customer and business data confidentiality and integrity, as the platform stores personal data, order details, and account credentials. This risk occurs when threats such as data breaches exploit vulnerabilities including insecure communication channels, weak server configuration, or unpatched software components(R1). Another major risk is the loss of administrative control over the platform, where unauthorized entities gain the ability to modify store configurations, products, pricing, or user accounts. This risk is driven by threats such as credential theft, brute-force attacks, or insufficient access restrictions, leading to serious operational and financial consequences (R1). In addition, Bagisto faces risks related to application manipulation through malicious input and

service unavailability, which may disrupt system integrity and business operations due to injection attacks, denial of service attempts, or improper system configuration (R4, R1)

2.2 Origins of These Risks

The origins of security risks in Bagisto are primarily linked to software vulnerabilities caused by outdated or unpatched components, as the platform depends on multiple dependencies and server components that require regular updates (R1). Another major origin is design weaknesses associated with high-privilege system components, particularly the admin panel, where concentrating critical functionality in a single interface increases risk if access controls are insufficient. Security risks may also arise from implementation flaws in input handling mechanisms, where inconsistent or insufficient validation and sanitization allow malicious input to be processed. Such weaknesses can enable attacks such as injection or cross-site scripting (XSS), compromising system integrity and user sessions (R4). Furthermore, continuous external threats targeting e-commerce platforms and operational misconfigurations, such as exposed services and weak firewall rules, significantly contribute to the overall risk landscape.

3. Data Protection

3.1 Data Types

Bagisto has normal shopping website properties ranging from highly confidential to public data collections. These data types include customer personal information from logins and personal accounts like name, phone number, email, address, age, and passwords, which are highly confidential / critical. Similarly, employee data defining their roles and responsibilities and all login information is also included and considered sensitive. Another data type includes product data, which can be both public and sensitive, such as product name, ID, price, importer, and stock quantity, which brings us to inventory data like stock quantity, location of each item, and building number that is generally sensitive. Lastly, it includes order and transaction data, which includes order status, order details (product, price, and location), order number, and payment details such as method, time, and record, which are considered critical.

3.2 Existing Protection Measures

The system has a built in access control layer which gives role based access control by dividing the administration with full access, employee with responsibilities specific access, and customer with browsing and ordering access (R2). The framework is built on open source laravel which gives features such as authentication, authorization, and validation(R3). Another essential feature is password protection and validation by hashing(R4).

3.3 Identified Weaknesses

The Bagisto website doesn't have HTTPS external configuration, which is considered one of the crucial protocols in security because it ensures that data is not compromised. There is also a lack

of two-factor authentication and additional restrictions for the admin area to ensure the area is not accessible from unauthorized locations. Manual protection is needed for highly sensitive files, especially in shopping apps, which include large amounts of sensitive data, so this must be implemented to ensure better security. Monitoring and logging throughout the application, even for guest users, is important to help detect suspicious activity, such as identifying the possible location of an attacker. Continuous logging should therefore be monitored to reduce or stop suspicious actions (R5).

4. Consequences of Security Failures

4.1 Immediate Consequences

When a security breach occurs for an e-commerce site, it can result in significant damage immediately. Customers sensitive data, such as phone numbers and order history can be exposed to the public. In addition, user credentials become compromised and misused by unauthorized individuals, and payment details are threatened by financial fraud. Also, the system may become unavailable or undergo a temporary shutdown, and customers will not be able to complete any transactions this impact the business operations and reduce productivity. Disruption in supply can cause delays in deliveries and disrupt stock logs, which cause financial implications and customer dissatisfaction (R6).

4.2 Long-term Consequences

The long-term consequences of security failures have a lasting negative impact on the business that can not be easily remedied. Privacy laws impose strict guidelines for data protection, and failing to meet these guidelines can result in severe penalties and legal consequences (R7). A company that is unable to protect its users' data and maintain confidentiality and integrity will result in a loss of customer trust and damage its reputation. Trying to overcome such a failure requires time and financial resources to investigate the breach, improve security measures, and restore the system and data. As other companies and users may be reluctant to continue using the platform, the revenue will decrease, and the company might go bankrupt.

4.3 Impact on Users

Security breaches direct impact on users include privacy invasion, as unauthorized access to personal data becomes available to anyone, which can lead to blackmail or stalking. Individuals become at risk of identity theft or financial fraud, and this can cause emotional distress. While indirect impact involves frustration due to the unavailable service, and having to update their login credentials. Also, users will most likely avoid using the system in the future due to the frequent issues. To ensure user satisfaction, businesses like Bagisto should implement safety measures throughout their system.

4.4 Impact on Company and Stakeholders

Stakeholders and the Bagisto company can be directly affected by security failures. Business owners can face financial losses due to the unavailable service to market their products. The company needs to be prepared for any potential lawsuit and to repay affected customers or partners. As for the indirect impact, stakeholders may lose trust in the system's reliability and security, which can harm future relationships and collaborations. Additionally, the loss of brand reputation and customer loyalty can negatively affect the company's name and marketplace, which makes difficult to attract customers and maintain existing relationships.

Team Contribution	
FayAlsubaie 222410396	Overview of the Software Domain
Norah Alfayez 222410215	Identification of Significant Security Risks
Layan Alhabib 222410739	Data Protection
Nadin Alaskar 222410587	Consequences of Security Failures

References

- R1-<https://devdocs.bagisto.com/getting-started/best-security-practice.html>
- R2-<https://devdocs.bagisto.com/package-development/access-control-list.html>
- R3-<https://laravel.com/>
- R4-<https://devdocs.bagisto.com/theme-development/validation.html>
- R5-<https://devdocs.bagisto.com/getting-started/best-security-practice.html>
- R6- [Owasp.org](https://owasp.org). <https://owasp.org/>
- R7- <https://nca.gov.sa/en/laws-and-regulations/>



Bagisto Project - Phase 2

Nadin Alaskar 222410587

Layan Alhabib 222410739

Norah Alfayez 222410215

Fay Alsubaie 222410396

College of Computer & Information Science

Secure Software Development CYS402

Dr. Zainab Masood

1. Product Assets

Our platform bagisto e-commerce, has many critical assets that must be protected from any possible targeted attacks. These assets include data assets, system assets, and intangible assets. Protecting these assets is important to ensure business operational continuity, prevent misuse of unauthorized access, and maintain the company's reputation and customer trust.

1.1 Asset Identification & Description

Resources

These are the tangible assets in the system, and they include:

- **Customer personal data-** Includes customers personal information like customer names, email addresses, phone numbers, shipping addresses, and order history.
- **Employee data-** Includes the employee's personal information, with some employee identification information as well, and some role-based capabilities that give access permission to specific parts of the system.
- **Authentication data-**Includes login credentials, hashed passwords, and session tokens.
- **Payment data-** Includes payment transaction details, billing information, and other records used to track transactions.
- **Order records data-** Includes order ID, purchased items, order status, and delivery location details.
- **Business configuration data-** Includes product inventory, discount codes, and system configuration parameters.
- **Inventory data-**Includes stock quantities, product availability status, warehouse location details, and inventory update records.

Functionalities

These include system functionalities that enable the platform to operate, and these include:

- **User registration and authentication-** This includes the customer and employee login, authentication, and validation to enter the system and perform desired actions.
- **Payment process:** This includes connecting to external gateways to ensure a successful payment transaction.
- **Order process-** This includes enabling customers to initiate and complete purchase transactions.
- **Inventory tracking and management-** This includes the ability of the system to manage and track inventory, remove/add inventory, and locate items.
- **Discount and pricing configuration-**This includes the ability to perform discount configurations enabled by administrators.

Intangible Assets

These are nonphysical assets that are important for the business, and they include:

- **System availability-** This includes ensuring that the platform remains accessible and operational for customers and employees without any downtime.
- **Company reputation-** This includes the company maintaining a public image that is perceived well by customers and partners, with traits such as reliability and security.
- **Customer trust-** This includes the confidence that customers have when sharing personal and payment information with the company.
- **Revenue stability-** This includes maintaining consistent financial performance and preventing financial losses that could result from security incidents or system failures.

1.2 Asset Prioritization

It is important to prioritize the assets based on their importance and the impact they have on the business, and this asset prioritization is performed by evaluating the sensitivity of the asset, the likelihood of it being targeted, and the impact on confidentiality, integrity, and availability.

High priority asset:

- **Authentication Data-** This asset is considered highly sensitive because login credentials are an attacker's main target, and compromising them may lead to full system access and privilege escalation. Also, if compromised, it violates the CIA triad by destroying confidentiality through unauthorized access, integrity through data manipulation, and availability through account lockout.
- **Payment data:** Any payment-related information is always considered highly sensitive due to the severe consequences if compromised. Since the impact is significant from a risk modeling standpoint due to financial fraud, legal liability, and reputational harm.
- **Customer personal data-** This asset is considered sensitive because it contains highly confidential information about the customer.
- **Intangible assets (availability, reputation, trust, revenue)-** All of these are highly important to prioritize because any downtime/ bad reputation causes loss of trust and revenue.

Medium priority asset:

- **Order records data-** Important for integrity and business operations, but the impact is more operational
- **Business configuration data-** Any manipulation done can affect the pricing and business rules, but the impact is moderate.

Low priority asset:

- **Employee data-** Internally significant, but typically less appealing to hackers than customer financial data.
- **Inventory data-** While manipulation may cause operational disruption and short-term financial loss, it does not typically expose sensitive personal or financial data.

1.3 Decision Process

The process of prioritizing was done based on a structured risk based approach. This approach followed a decision process individually on each asset based on its exposure level, potential impact if exploited, and business value.

The following criteria were applied in this decision-making process:

- **Sensitive of the asset-** Meaning the information/data in the asset how sensitive is it? Does it contain confidential or financial information? If yes, it is considered a critical and high-priority asset with severe consequences if disclosed.
- **Impact on CIA triad-** Each asset could use this triad to evaluate how it would affect each of confidentiality, integrity, and availability if compromised.
- **Likelihood of being targeted-** This refers to how attractive an asset is to an attacker. Attackers are most likely to target assets that provide high value, such as financial gain, unauthorized access, or sensitive information
- **Business impact-** Some assets are important, and they directly affect revenue and customer trust. Because security needs to support business objectives, it is ranked higher.

By combining these criteria, assets with a high likelihood of exploitation and severe business or security consequences were classified as high priority. Assets with moderate operations impacts were classified as medium priority. Assets with limited or low business and security consequences were classified as low priority. All of this ranking is based on the $\text{Risk} = \text{likelihood} \times \text{consequences}$. Following this structure for the decision process is essential to classify your assets and implement the security required for each one from an early stage, following the secure SDLC principles.

1.4 Future Changes

As the Bagisto platform evolves, the systems assets and their prioritization might change over time to keep up with the new business growth and system updates. Once the system scales, the number of users will increase and require more storage space due to the large amount of transactions and authentication of user credentials. As the data grows, so will the risk associated with data breaches, making customers and payment data a higher priority asset that needs to be protected using strong and secure methods. Also, adding new features such as new payment gateways, third party integrations, or mobile applications that introduce new data and endpoints. These additional features can increase the attack surface, allowing more attackers in than usual. Therefore, asset identification and prioritization need to be reviewed and updated regularly to ensure that the system grows appropriately.

2. Malicious Actors & Threats

This section aims to identify the attackers and threats of the system. Understanding the attacker, their skills, and their motivation is useful in establishing threat modeling. These steps help in identifying the appropriate security controls and protect the systems assets.

2.1 Types of Attackers

Attacker Type	Skill Level	Motivation	Example
External attackers	Medium to high	- Financial gain - Data theft	Exploiting web vulnerabilities to steal customer data or perform fraudulent transactions
Malicious insiders	High	- Abuse of privileges - Personal gain	Employee misusing admin access to modify prices or access sensitive data
Automated bots	Low to medium	- Mass exploitation - Credential abuse	Credential stuffing attacks using leaked username & password lists
Low skill attackers	Low	- Curiosity - Reputation	Using publicly available tools to test for SQL injection or XSS
Competitors	Medium	- Competitive advantage	Collecting product data or pricing information

2.2 High-Level Threats (STRIDE)

STRIDE modeling helps discover and evaluate high-level threats that may affect the Bagisto eCommerce platform. The threats stem from the system's authentication, administrative features, the checkout process, and how data is managed.

Spoofting

Spoofing is an incident when an attacker masquerades as a genuine user. In Bagisto, this threat would occur when an attacker impersonates a customer or admin user and logs in as one of them. Given that there are different authentication guards for customers and administrators in Bagisto, an attacker with access to a user's account would gain access to sensitive functionality in the store admin, or worse, the admin features.

Tampering

Tampering is the unauthorized alteration of data. In Bagisto, an attacker could manipulate a checkout or cart-related request to alter one of the order-related items and, say, change the shipping or payment option. This could result in the order being processed contrary to the business rules, and this would affect the correctness of a transaction as well as the integrity of the system.

Repudiation

Repudiation is an act where a user tries to distance themselves from an action. In Bagisto, an administrative user might try to distance themselves from making critical changes like refunds, order cancellations, or changes to system configurations. This is possible if there is no detailed single audit log. A detailed audit log is something that is missing in the system because there is some form of logging gone with no specific audit log for the actions taken of those with higher/higher access. This increases the risk for impersonation/repudiation claims to be made.

Information Disclosure

Information disclosure is the act of giving access to the sensitive documents of the system to an unauthorized person. Bagisto stores sensitive data such as the personal and order data of the customers. This data could be unintentionally exposed through the APIs, caches, or system errors if proper access controls are not implemented or cache is not configured properly. This is a clear breach of customer privacy.

Denial of Service (DoS)

Denial of Service threats are to lower or eliminate the availability of the system. A potential DoS threat in Bagisto is when a large number of requests are made on an endpoint that consumes a large number of resources like login, password reset, checkout or search. Some form of throttling has been implemented on the password reset functionality, but all other public endpoints have the potential to be affected by an excessive load.

Elevation of Privilege

Elevation of Privilege in the context of Bagisto happens whenever a user obtains access to more roles or permissions than they were originally intended to have assigned. Administrative access in the system for Bagisto is facilitated by roles and access control list (ACL) mapping. In some cases, if there are any configuration errors in the permissions, or if there are any gaps in the mapping of routes to permissions, then admins with employee-level access can exploit the system to access other administrative functions that are beyond the scope of their assigned roles.

2.3 CIA Mapping

Mapping the described STRIDE threats to the CIA triad helps to clarify the threats in terms of confidentiality, integrity, and availability.

Threat	Confidentiality	Integrity	Availability	Explanation
Spoofing	Yes	Yes	No	Admin or user accounts that have not been secured can be accessed by an attacker to perform functions under the identity of the user or admin.
Tampering	No	Yes	Sometimes	Order or checkout data that have not been secured can be modified by an attacker. This affects integrity and can be a nuisance, or a major disruption to a system.
Repudiation	No	Yes	No	Insecure audit trails lead to a loss of confidence in the integrity and accountability of the admin actions.

Information Disclosure	Yes	No	No	Customer and order information that have not been secured can be disclosed, which is a breach of confidentiality.
Denial of Service	No	No	Yes	Denial of service attacks can be carried out by an attacker to render the system available to legitimate users.
Elevation of Privilege	Yes	Yes	Sometimes	Data that is meant to be restricted and the system itself can be modified by an attacker to unacceptably influence administrative functions. This is the result of permission that is greater than what is appropriately allowed.

2.4 Available Information to Attackers

- **Public pages:** Product listings, categories, prices, and general website structure that can be seen by anyone and used to understand the system behavior and entry points.
- **API responses:** Endpoint structures, request parameters, and response format can be revealed by unsecured APIs, and attackers can analyze these responses to find out the weaknesses.
- **Error messages and system feedback:** Can expose database behaviour, framework details, or configuration information, which an attacker can use to craft their own attack.
- **Metadata:** Technical information such as HTTP headers, cookies, and session identifiers can reveal authentication procedures and session management.
- **Social engineering data:** Public information about employees, administrators, users, and their behaviour can be used to perform phishing or targeted attacks.

2.5 Exploitation Methods

As an eCommerce platform and user transactional services, Bagisto is vulnerable to many (and many types of) compromises. These attacks tend to be 'simple' in nature and are the result of the exploiter's utilization of poorly designed (or under-conceived) web applications, poor user behavior, and/or poorly configured web applications.

SQL Injection (SQLi)

Web applications that allow users to enter data into form fields or any input that the user controls and are connected to a backend database (like Bagisto) are susceptible to SQL injection attacks. In an instance of a web application (i.e. Bagisto) there are enough user input fields that the web application can be susceptible to the attack(s): search input, filtering input, and fields that capture/collect/hold/ or define user shipping address and coupon code usage/ discounts (i.e. storefront and API calls to the underlying database). While (in this instance) the web application can be susceptible to attacks because the underlying database is not able to have the developer use policies as a way to prevent SQL injection attacks as the developer needs to write code - attacks are possible in the areas where the developer has chosen to allow the database to be accessible from the application code via the use of SQL (i.e. Dynamic SQL). This creates the potential loss of the developer's ability to prevent SQL injection attacks. In Bagisto, the developer (in this instance) has lost the ability to fully secure the application because the framework that is supported is Laravel's Eloquent ORM and Query Builder, because they provide parameterized queries by default.

Cross-Site Scripting (XSS)

When someone places a bad script into a webpage, that is called cross-site scripting. Bagisto shows content that comes from admins or users, like the product descriptions, CMS pages, and marketing content. If that content is not sanitized frequently, it can cause session hijacking or counterfeit user interfaces. While Bagisto does contain security headers and content sanitization security measures, XSS is still a problem if not all views are protected the same way.

Phishing

Phishing is when an attacker tries to get a user's personal information instead of going after a system. Bagisto has a password reset, account verification, and social login process that all send users links and direct them to pages for user authentication. Because of these features, user education and safe messaging is important, as attackers can spoof legitimate emails or login pages.

Weak Authentication

Weak mechanisms of authentication raise the chances of account breach. Although Bagisto uses separate guards for users and admins and uses password hashing, the password requirements are very basic. Also, the fact that there is a default admin account at install indicates poor initial setup and password management. Also, poor initial setup and password management are the reasons that the importance of multi-factor verification is added

Brute Force Attacks

Brute force attacks are attacks that repeatedly try to login or change password to gain access to a system or to drain system resources. In Bagisto, the endpoints for login and password reset are public. Although some authentication processes have CAPTCHA and throttling, there is no consistent rate limiting to allow some endpoints, which could result in a wide open door for automated attacks.

Misconfiguration

Misconfiguration is a frequent cause of unsafe situations. In Bagisto, the security is in the hands of the user based on the system's correct environmental and deployment configurations, such as secure session cookies, roles and permissions, and the state of the installer. A user with the role of a misconfigured administrator can bring remote management. and unprotected CSRF payment endpoints require the user to be very securely deployed.

3. Software Requirements (SQUARE-Lite Section)

3.1 Agree on Definitions

We have used the following definitions during this phase to make sure that the security requirements process is clear:

Threat: The treats are event or action that could cause harm to the system or its users. In Bagisto, an unauthorized attempt to access the admin account or disrupt checkout functionality are some example of it.

Vulnerability: The vulnerability is some weakness in the system that can be exploited by a threat. Some examples for the vulnerability are missing the validation for some inputs and misconfigured some permissions.

Risk: The risk can be defined as a threat that exploits a vulnerability and what would happen if it occurs. In Bagisto, the risks are higher when it comes to important data like the customer data, the orders, or the administrative controls.

Security Requirement: This is a rule or condition that the system must have to make the security risks less. For example the role-based access controls should be enforced and limiting the number of the attempts to do authentication.

3.2 Identify Security Goals

We have identified the following security rules based on the functionality of the platform Bagisto:

1. Protect the confidentiality of the customers by making sure that the personal data like the profiles, addresses, and the history of the orders are seen only by the authorized users.
2. Maintain the integrity of the transactions so no one unauthorized can change the contents of the cart, the prices, the discounts and the shipping costs.
3. Prevent access that is unauthorized to the administrative features. This is done by forcing strong authentication and access based on role in the admin panel.
4. Make sure the critical services are available all the time even in the times of high traffic. This includes the checkout, payments and the access to the accounts.
5. Protect the information about the payments by making the payment callbacks secure and keep the transaction records accurate.
6. Provide responsibility for the actions that need privileges by recording the logs for the important events like logins, changing the orders, and refunds.

3.3 Elicit Security Requirements

ID SR-##	Author	Requirement	Category	SMART
SR01	Layan	The system shall enforce multi-factor authentication for all employee accounts.	Authentication	Specific: Applies to all employee accounts in the system. Measurable: Access is granted only after successful verification of at least two authentication factors. Achievable: Can be implemented using OTP. Relevant: Protects privileged accounts from unauthorized access.

				Time-bound: Must be implemented before system deployment.
SR02	Layan	The system shall store all user passwords using a secure hashing algorithm	Data protection	Specific: Applies to all stored passwords. Measurable: Passwords must not be stored in plain text. Achievable: Hashing libraries are available in the framework. Relevant: Protects authentication data confidentiality. Time-bound: Must be enforced during system implementation phase.
SR03	Layan	The system shall implement role-based access control to restrict access to system functionalities based on user roles.	Access control	Specific: Access permissions defined by role. Measurable: Users cannot access unauthorized systems. Achievable: Can be configured within application framework. Relevant: Prevents privilege escalation. Time-bound: Must be configured before user account activation
SR04	Layan	The system shall automatically lock an employee account after 5 consecutive failed login attempts.	Authentication	Specific: Applies to all employee login attempts. Measurable: Account is locked after exactly 5 failed attempts. Achievable: Can be implemented through login attempt. Relevant: Prevents brute-force attacks. Time-bound: Must be configured before system deployment.
SR05	Layan	The system shall encrypt all sensitive data transmitted between users and the server	Data protection	Specific: Applies to all sensitive data transmissions. Measurable: No

		using HTTPS		sensitive data shall be transmitted over unsecured HTTP. Achievable: Can be implemented using SSL/TLS certificates. Relevant: Protects confidentiality and integrity of user and payment data. Time-bound: Must be enforced before production release.
SR06	Fay	The system shall enforce a minimum 8-character password requirement for all customers and admin at account creation and password reset.	Authentication	Specific: Password length Measurable: Minimum 8-character Achievable: Supported by Laravel validation Relevant: Lowers account takeover Time-bound: Enforced at time of password submission
SR07	Fay	The system shall block a customer or admin account for 15 minutes after 5 failed log-in attempts.	Authentication	Specific: Log-in attempts limit Measurable: 5 attempts, 15 mins locked Achievable: Supported by Laravel throttling Relevant: Stops brute-force attacks Time-bound: Immediate at threshold
SR08	Fay	The system shall allow access to admin panel routes only to authenticated users, and only to those with the appropriate	Authentication	Specific: Access to admin routes Measurable: Authentication + check permission Achievable: Via guards

		role-based access.		and ACL Relevant: Stops privilege escalation Time-bound: On every admin request
SR09	Fay	The system shall ensure that customer personal data (addresses and order history) can be accessed only by the owning customer and authorized admin users.	Data protection/ Access control	Specific: Customer personal data Measurable: Restricted by user identity and user role Achievable: Encouraged by middleware and guidelines Relevant: Maintains confidentiality Time-bound: Requested on all data
SR10	Fay	The system shall validate and sanitize all user input in storefront pages to prevent the execution of malicious content.	Input validation	Specific: Input sanitization Measurable: Visible user/admin content on all pages Achievable: Utilizes existing sanitization tools Relevant: Lowers XSS risk Time-bound: Before content is rendered
SR11	Norah	The system shall log all admin login attempts (successful and unsuccessful) and record both the timestamp and user ID associated with each login attempt.	Logging & Monitoring	Specific: Admin login events Measurable: Success and failure log Achievable: Laravel logging supports it Relevant: Accountability and investigations

				Time-bound: Time of log = time of login event
SR12	Norah	The system shall document all actions taken by an admin with special privileges that include but are not limited to the cancellation of orders, the issuing of refunds, and the making of changes to the system's configurations.	Logging & Accountability	Specific: Privileged admin actions Measurable: Action, user, and timestamp are recorded Achievable: Action logging can provide this Relevant: Prevents repudiation Time-bound: When action takes place, timestamp is recorded
SR13	Norah	The system shall include CAPTCHA verification on the customer login and password reset forms after several unsuccessful attempts.	Authentication / Abuse Prevention	Specific: Use of CAPTCHA Measurable: After series of failed attempts, this will happen Achievable: Use of CAPTCHA is supported Relevant: Automated abuse will be reduced Time-bound: This will happen during the authentication process
SR14	Norah	The system shall include a provision that ensures the payment reply endpoint can only be called by certain specified, authorized payment providers.	Payment Security	Specific: Payment callbacks Measurable: Source validation will be enforced Achievable: This is supported with gateway verification Relevant: Prevents misconfiguration abuse Time-bound: Enforced after deployment
SR15	Norah	When the application is deployed to the production, the system shall disable the installation and setup routes.	Configuration & Deployment	Specific: Installer access Measurable: Routes disabled after setup Achievable: Controlled

			Security	by middleware Relevant: Prevents misconfiguration abuse Time-bound: Enforced after deployment
SR16	Nadin	The system shall limit the number of requests to public API endpoints to prevent DOS attacks	Availability	Specific: Applies to all public APIs Measurable: Requests exceeding the limit are blocked Achievable: Supported by server configuration Relevant: Mitigates DOS Time-bound: Enforced before production deployment
SR17	Nadin	The system shall mask sensitive information in all error messages returned to users and APIs	Information disclosure	Specific: Applies to error messages only Measurable: No database or system details exposed Achievable: Supported by framework error handling Relevant: Prevents information leakage Time-bound: Enforced during application runtime
SR18	Nadin	The system shall automatically invalidate user sessions after a defined period of inactivity	Session management	Specific: Applies to all authenticated sessions Measurable: Sessions expire after inactivity timeout Achievable: Supported by session configuration Relevant: Reduces session hijacking risk Time-bound: After user login
SR19	Nadin	The system shall restrict administrative access to the management panel to trusted IP addresses or network ranges	Access Control	Specific: Applies only to admin panel access Measurable: Access denied from untrusted IPs

				Achievable: Supported by firewall Relevant: Protects important functionality Time-bound: Configured before system deployment
SR20	Nadin	The system shall log and monitor all failed payment attempts and trigger alerts when abnormal patterns are detected	Logging & Monitoring	Specific: Applies to payment failures Measurable: Logs include timestamps and user identifiers Achievable: Supported by logging and alerting tools Relevant: Detects fraud and abuse Time-bound: Active during payment processing

3.4 Prioritize Requirements

Requirement ID	Priority	Reason
SR01	High	To protect employees accounts from unauthorized access.
SR02	High	To prevents credentials from being compromised by securely storing passwords.
SR03	High	To restrict unauthorized access based on user rules.
SR04	High	To reduce brute force attacks on employees accounts.
SR05	High	To ensure the secure transmission of sensitive data over the network.

SR06	Medium	While stronger passwords may improve security, other measures like rate limiting and access control are far more effective at avoiding immediate security compromises.
SR07	High	The account lockout feature directly mitigates the risk of brute force and credential stuffing attacks that are routinely encountered in eCommerce systems.
SR08	High	Limiting admin access is vital, so that unauthorized control over products, orders and configurations is avoided.
SR09	High	Because of the impact of data breaches and data privacy legislation, the risk to customer data must be minimized.
SR10	Medium	Although input sanitization mitigates risk of XSS, the impact is usually far less than that of authentication or authorization failures.
SR11	Medium	While login logs do not help directly avoid attacks, they assist in investigations and accountability.

SR12	High	Without logging, refusal events, and negligent use of admin rights can occur, so logging admin activities is critical
SR13	Medium	Although CAPTCHA is helpful in reducing automated attacks and abuse, it is not as important as some form of access control and account lockout.
SR14	High	To avoid losses and fraud, it is of utmost importance to secure callbacks of payments.
SR15	High	Once installed, routes and installers should be disabled to prevent serious misconfigurations and changes to the system that are not authorized.
SR16	High	Prevents service disruption that affects system availability.
SR17	High	Reduces information leakage that could help attackers exploit the system.

SR18	Medium	Lowers session hijacking risk but has less impact than user authentication.
SR19	High	Protects highly important admin functions from unauthorized access.
SR20	Medium	Helps detect fraud but does not directly prevent attacks.

4. Team Contribution Table

Team Contribution	
FayAlsubaie 222410396	5 security requirements 2.2 High-Level Threats (STRIDE) 2.3 CIA Mapping Reviewed: Norah Alfayez
Norah Alfayez 222410215	5 security requirements 2.5 Exploitation Methods 3.1 Agree on Definitions 3.2 Identify Security Goals Reviewed: Layan Alhabib
Layan Alhabib 222410739	5 security requirements 1. Product Assets 1.1 Asset Identification & Description 1.2 Asset Prioritization 1.3 Decision Process Reviewed: Nadin Alaskar
Nadin Alaskar 222410587	5 security requirements 1.4 Future Changes 2. Malicious Actors & Threats 2.1 Types of Attackers 2.4 Available Information to Attackers Reviewed: FayAlsubaie

References

<https://devdocs.bagisto.com/getting-started/best-security-practice.html>

<https://devdocs.bagisto.com/package-development/access-control-list.html>

<https://devdocs.bagisto.com/theme-development/validation.html>

<https://devdocs.bagisto.com/getting-started/best-security-practice.html>



College of Computer & Information Science

Secure Software Development

CYS402

Bagisto Project - Phase 3

Dr. Zainab Masood

Nadin Alaskar 222410587

Layan Alhabib 222410739

Norah Alfayez 222410215

Fay Alsubaie 222410396

Architecture Overview

System Architecture Overview

Bagisto is an open-source eCommerce platform that uses a modular monolithic application, built using the Laravel framework. They use a package-based system, ensuring that each primary function is kept within its own individual module, found at the packages/Webkul directory.

The system is divided into three overarching sections.

Applicable to this study are the first two sections that are referred to as the primary domain, administrative function, and storefront:

- **The Storefront Layer:** Manages all customer-related activities like product browsing, cart management, and checkout, and purchase completion.
- **The Admin Layer:** Encompasses administrative functions like order, product, customer and system configuration management.
- **Core Domain Layer:** Encompasses the business logic modules, like catalog, checkout, payment, and customer management.

The separation boosts modular development and is accompanied by improved maintainability.

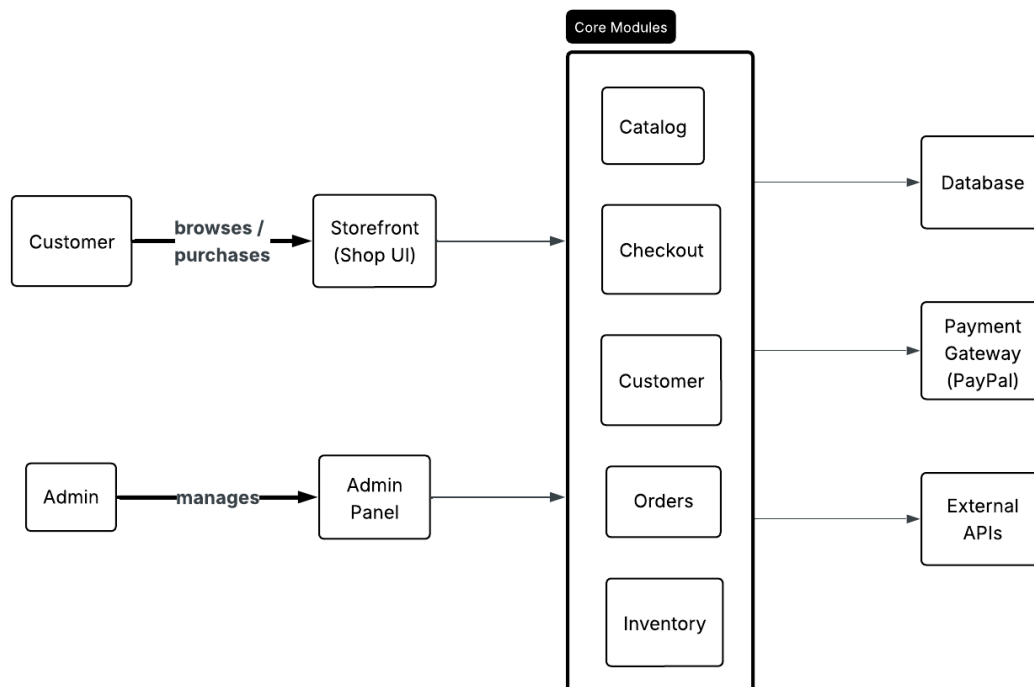


Figure 1 High-Level Architecture of Bagisto (Created using Lucidchart based on Bagisto source code)

This system is designed using a layered Model-View-Controller architecture. User interaction is managed at the View layer, which includes the Storefront and Admin Panel. The Laravel backend and controllers construct the Controller layer, performing request processing and business logic execution. The Model layer is composed of the database and core modules to handle data and persistence. This design divides and decreases system complexity, which also enhances maintainability and security.

Subsystems Description

The architecture of Bagisto is made up of several subsystems. Below, each identified subsystem will be briefly described.

1. Admin Panel

The Admin Panel subsystem represents back-office functions like product, order, customer and configuration management.

The subsystem is highly sensitive due to the fact that it handles core regulatory functions of the system.

The subsystem is housed in:

- packages/Webkul/Admin

2. Storefront (Shop UI)

The Storefront subsystem is in charge of customer activities, including product browsing, account management, and checkout.

The subsystem is housed in:

- packages/Webkul/Shop

3. Storefront API

The Storefront API subsystem provides a number of RESTful APIs to facilitate frontend activities like cart management, checkout and customer authentication.

The subsystem is housed in:

- packages/Webkul/Shop/src/Routes/api.php

4. Catalog Management

The subsystem is responsible for product, category and attribute management.

The subsystem is housed in:

- packages/Webkul/Product
- packages/Webkul/Category
- packages/Webkul/Attribute

5. Checkout and Sales

The subsystem is responsible for cart management, order processing, invoicing, and refunds.

It is implemented within:

- `packages/Webkul/Checkout`
- `packages/Webkul/Sales`

6. Customer Management

Handles customer accounts, authentication, profiles, and private information.

It is implemented in:

- `packages/Webkul/Customer`

7. Admin Users & Roles (Security Subsystem)

Admin authentication, role assignments, and permission assignments through ACL.

It is implemented in:

- `packages/Webkul/User`

This is a fundamental security subsystem.

8. Payment Subsystem

Responsible for payment methods and external payment integration like PayPal.

Implemented in:

- packages/Webkul/Payment
- packages/Webkul/Paypal

9. Shipping Subsystem

Responsible for managing shipping methods and calculating rates.

Implemented in:

- packages/Webkul/Shipping

10. Core Platform Services

Provides shared services like configuration, inventory, tax, and data management.

Implemented in:

- packages/Webkul/Core

11. CMS & Theme

Responsible for content management and rendering at the frontend.

Implemented in:

- packages/Webkul/CMS
- packages/Webkul/Theme

12. Marketing & SEO

Responsible for managing promotions, campaigns, and SEO configuration.

Implemented in:

- packages/Webkul/Marketing

Security Specific Subsystems

The system has specific security features:

- **Authentication System**
 - o Uses Laravel guards (admin, customer)
 - o Session-based authentication
- **Authorization (ACL)**
 - o Role-based access for admin users
- **Middleware Security**
 - o Authentication middleware
 - o Secure headers middleware
- **CAPTCHA**
 - o Protects the login and registration flows
- **Session & CSRF Protection**
 - o This is built into the Laravel framework by default.

Security Risks Analysis

The various subsystems in Bagisto tend to present unique security challenges based on their function, exposure, and sensitivity of data handled.

Admin Panel

The Admin Panel is the most important subsystem because of its involvement in more sensitive activities such as product management, pricing, and user management. The biggest security threats in this module are broken access control, failure to authorize, and administrative abuse.

When access control and ACLs are left unconfigured access control, they may be opened to lower privilege users. Furthermore, user input and file upload mechanisms are exploited by stored Cross-Site Scripting (XSS) and arbitrary file execution.

Gaining control over this subsystem will allow a threat actor to control the entire system, thereby modifying orders, customers' personal data, and the entire system of financial transactions.

Storefront

The Storefront subsystem is responsible for all interactions with the customers, and in doing so, it handles a lot of untrusted data. The primary risks are the system's failure to validate its input, poor management of client sessions, and abuse of the business logic tier.

The system may be exposed to user-generated content if a developer fails to sanitize them, thus creating a mechanism for reflected/stored XSS. Insufficient rate limiting is also a threat to the system and may initiate impersonation of users.

Attackers, once they take control of Storefront, will be able to steal customer accounts, abuse the system to modify customer orders, and inject a virus to all users.

API Layer

The API subsystem is the most exposed part of the system because its underlying functionality can be reached programmatically, which makes it highly prone to access control and validation breaches.

Common risks consist of broken object-level authorization (IDOR), overexposed data, and request throttling (or lack thereof). Developers may forget middleware, or may leave authorization checks incomplete, thereby permitting access to sensitive operations to unauthorized users.

An exposed API may enable attackers to retrieve and manipulate data, resulting in massive data breaches and perpetrating fraud on an industrial scale.

Authentication System

The authentication subsystem deals with user identity verification and access control based on Laravel guards and session management.

The absence or improper configuration of authentication guards and poor session management can lead to unauthorized access to user accounts and session hijacking, session fixation attacks and access to customer and admin accounts. This, in turn, may lead to a complete compromise of the system.

Payment Subsystem

The payment subsystem is responsible for the transaction processing and the integration of the system with the external payment gateways.

Some of the risks involved in this subsystem include improper validation of payment callback, misplaced trust in the data provided by the client, and insufficient verification of the requests. In this subsystem, developer mistakes such as failure to validate the authenticity of the webhook may lead to loss of money.

The compromise of this subsystem leads directly to financial loss, creation of fraudulent orders and damage to business integrity.

Checkout System

The checkout subsystem is responsible for order processing, pricing and transaction logic.

Some of the risks include pricing logic manipulation, race conditions, and improper validation of cart data. Mistakes by developers such as trusting client-side values or failing to revalidate totals can lead to undercharging, abuse of discounts, and unjustly forcing the system to incur costs.

The compromise of this subsystem can lead to financial inconsistencies, fraudulent transactions, and customer dissatisfaction.

Catalog System

The catalog subsystem is responsible for managing product data and the content shown to users.

It is particularly vulnerable to stored XSS attacks. Because of the rich content fields, unsanitized product descriptions or CMS content can allow an attacker to inject malicious scripts.

A breach can lead to ongoing assaults that impact every user, as well as causing harm to the platform's reputation.

Impact of Subsystem Compromise

Because of its system-wide interdependence, a breach of a single subsystem results in a breach of the entire system.

- Breach of Admin Panel / Authentication systems means the system can be fully controlled.
- Breach of Payment / Checkout systems means fraudulent finances and breach of data integrity.
- Breach of Storefront / Catalog systems means users will be exposed to malicious content and theft of confidential data.
- Breach of API systems means large scale automated attacks can be executed.

Subsystems Susceptibility to Vulnerabilities

From the review, subsystems can be categorized as;

- High Risk
 - Admin Panel
 - Authentication System
 - Payment Subsystem
- Medium Risk
 - API Layer

- o Checkout System
- Lower Risk (but still important)
 - o Storefront
 - o Catalog System

Security Architecture Evaluation

Several security measures are incorporated in Bagisto's architecture

- Defense in Depth
 - o This is protective measures layered in the system, such as authentication, middleware, and access control.
- Role-Based Access Control (RBAC)
 - o Administration access is limited to the defined roles and authority.
- Secure Defaults
 - o Out of the box, Laravel offers built safeguards such as CSRF protection, password hashing, and session security.
- Separation of Concerns
 - o The distinction of Admin functions and customer functions reduces risk exposure.

The system operates solely based on the correctness of configuration and implementation by the developer. No configuration, or incorrect implementation of validation may lead to high susceptibility to unforeseen risks in the presence of security measures.

Threat Model

Overview

Using the OWASP Threat Dragon, we can build a model to illustrate, identify, and analyze possible security threats to the Bagisto system. This model uses the STRIDE framework to illustrate the components of the system, the data flows, the point of trust and the boundaries of possible threats.

The diagram shows the different users of the system and the methods of the attacks.

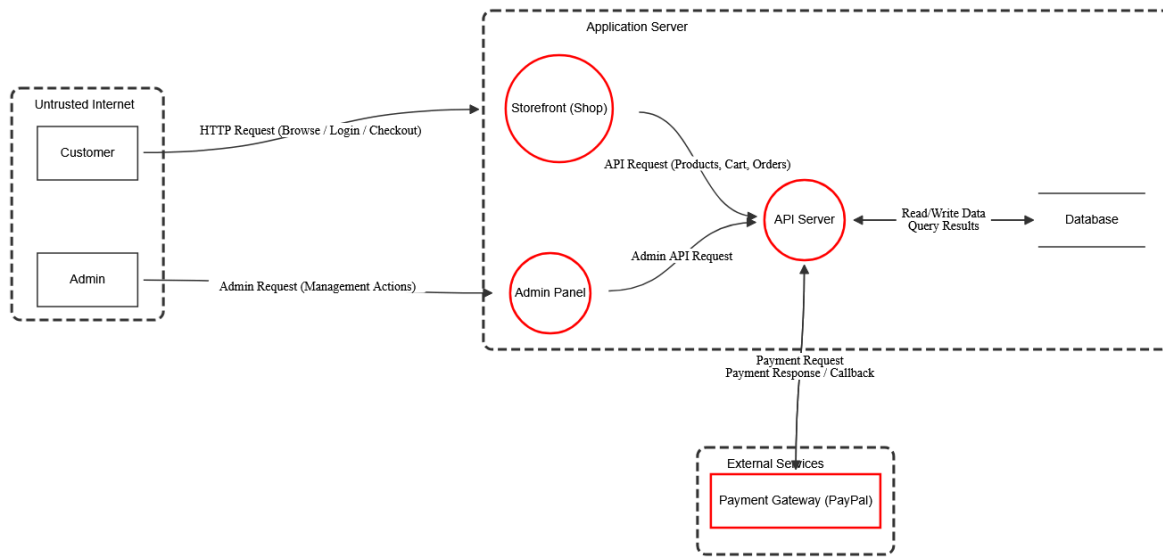


Figure 3 Threat Model Diagram (Generated using OWASP Threat Dragon)

Threats for the Storefront

Threats

#1 Cross-Site Scripting (XSS)
Information disclosure



STRIDE

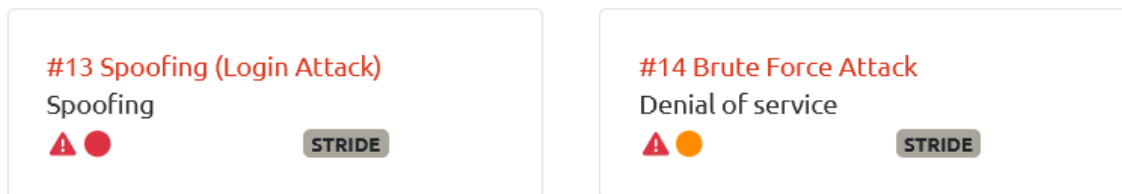
#2 Input Manipulation
Tampering



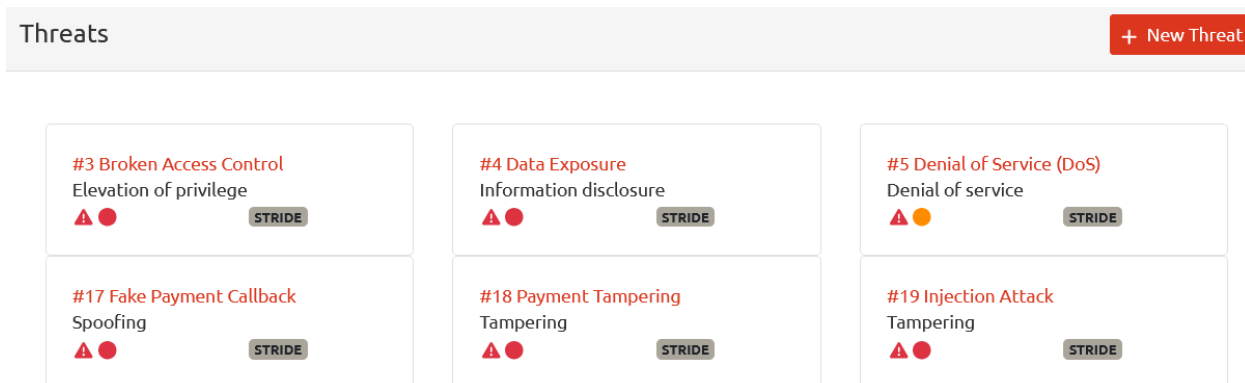
STRIDE

Threats for the Admin Panel

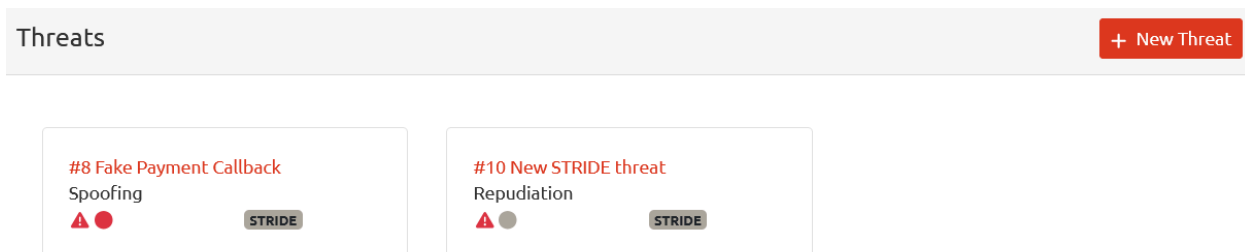
Threats



Threats for the API Server



Threats for the Payment Gateway



Diagrams of the threat model feature various threats and the strategies used to mitigate each. Using the OWASP Threat Dragon tool, the figure below shows these threats and their corresponding mitigations. Below, for brevity, are some of the important threats and their respective mitigatory strategies.

Threats and Mitigations Summary

Threat	Subsystem	Mitigation
--------	-----------	------------

XSS Attack	Storefront	Input validation and output encoding
Privilege Escalation	Admin Panel	RBAC and ACL enforcement
API Data Exposure	API	Authentication and access control checks
Payment Callback Forgery	Payment	Signature validation and request verification
DoS Attack	API	Rate limiting and throttling

Machine Boundaries

The system describes several different operational environments or boundaries of machines:

- **Client Side (Untrusted Internet)**

This includes end users, customers, and administrators. Everyone in this area is outside of the system and is untrusted.

- **Application Server**

This is the main processing component of the system and includes:

- Storefront
- Admin Panel
- API Server

All the core logic and processing of requests happen within this boundary.

- **External Services**

This includes third-party services, for example, the payment system (e.g. PayPal), which are independent of the system and communicate with it via API.

The boundaries can change in different deployments (cloud vs local), but the logic separation does not change.

Trust Boundaries

Trust boundaries are the points within a system where the level of trust is different:

- **Internet Boundary**

- Separates the users, customers/admins, and the application server.
- A high-risk boundary where the majority of the attacks come from.

- **Application Boundary**

- Internal communication within components of the system.
- The boundary is still the area of concern.

- **External Services Boundary**

- Interaction with third-party services, like payment gateways.
- External systems are not to be trusted.

These boundaries are of utmost importance for determining where security measures are to be put in place.

Threat Identification

Threats were identified using the STRIDE model:

Spoofing

Attackers may impersonate legitimate users by stealing credentials or session tokens, especially in login systems.

Tampering

Data such as order details or payment information may be modified during transmission or processing, leading to incorrect transactions.

Repudiation

Lack of proper logging may allow users or administrators to deny performing critical actions such as refunds or configuration changes.

Information Disclosure

Sensitive data such as customer information or payment details may be exposed through insecure APIs or misconfigured responses.

Denial of Service (DoS)

Attackers may overload the system with requests, affecting availability and preventing legitimate users from accessing services.

Elevation of Privilege

Users may gain unauthorized access to higher-level permissions due to misconfigured roles or access control mechanisms.

Threat Mitigation

To reduce identified risks, the following mitigation strategies are applied:

- Strong Authentication & Authorization
 - Role-Based Access Control (RBAC)
 - Secure session management
- Input Validation & Output Encoding
 - Prevents injection and XSS attacks
- Secure Communication
 - Use of HTTPS for all data transmission

- API Security Controls
 - Rate limiting and access validation
- Payment Security
 - Verification of payment callbacks and transactions
- Logging and Monitoring
 - Ensures accountability and supports incident detection

Risk Concentration

The highest risk areas identified in the system include:

- Authentication and Admin Panel (full system access risk)
- Payment and Checkout subsystems (financial risk)
- API layer (data exposure and automation risk)

These areas require stronger security controls and continuous monitoring.

Asset to Threat Model Mapping

Overview

This part connects some key system assets listed in Phase 2 with their locations in the Bagisto architecture. The purpose is to show how sensitive data is stored and accessed and how data flow affects system security.

This analysis deals with five categories of assets.

- Customer Authentication Data
- Payment Data
- Personal Data of Customers
- Records of Orders
- Inventory Data

Authentication Data

Personal Data of Customers includes usernames and passwords, session tokens, and password reset links.

It is predominantly stored in the database system in the tables of customers and admins, as well as the password reset tables. It goes through the authentication guards of Laravel and some middleware of the customer and admin subsystems.

The admin subsystem and admin ACL are in control of the data.

From a security point of view, it is a very sensitive and centralized asset. Authentication, session control, and access control weaknesses may facilitate data breach. Since admin and customer

authentication rely on similar configurations, wrong configurations may create cross-role vulnerabilities.

Payment Data

This includes the records of transactions, some details of how transactions are processed, and the metadata of payment gateways.

It can be found in the tables of `order_payment` and `order_transactions` in the Sales subsystem, and it goes through the Checkout subsystem, Payment modules, and many others, even external ones like PayPal.

The way payment data is processed is risky, especially on the trust boundaries like external calls back PayPal IPN. If the validation of the endpoints is weak, false payment confirmations can be made.

Sales subsystem centralizes storage, but processing is distributed over multiple components. This makes this asset particularly sensitive to integration and logic errors.

Customer Personal Data

Customer Personal Data includes personally identifiable information (PII) such as names, emails, phone numbers, and addresses.

This data is processed through the Storefront (customer account operations) and Admin subsystems.

Having multiple subsystems with customer data increases the possibility of customer data breaches. API response, admin dashboards, and data export features all present customer data exposure risks, without proper access control, data filtering, and access control.

This asset is semi-concentrated. It is mostly located in the Customer subsystem, while some presence is found in the Sales subsystem.

Order Records

Order records include details of the purchases made, the items, the payments, the shipments, and the refunds, if any.

They reside in the Sales subsystem and span several tables, such as `orders`, `order_items`, `invoices`, and `shipments`. These tables are created during checkout and are visible to customers as well as admin.

This asset is heavily concentrated, as it is one of the most vital constituents of the business. Its compromise creates a significant detrimental effect on the financial record accuracy, order fulfillment, and the customers' confidence.

Order records have very high requirements for integrity and confidentiality, due to the presence of financial and personal data.

Inventory Data

Inventory data includes details regarding the quantity of products in stock, the warehouses, and the products that are available.

This data is stored in the tables related to inventory and is managed via the Product and Inventory subsystem. It is accessed during the browsing of the catalog, during the checkout validation, and during administrative tasks.

Inventory data is necessary for the integrity of the entire system, and while the data may not be as sensitive as other pieces of information, the value of the information being stored is also susceptible to inaccuracies and manipulation. Inventory values that are incorrect and/or manipulated can result in overselling, anomalies in the inventory stock, and/or can create denial of services at the enterprise level.

This asset is partially centralized in storage, but is also distributed across several subsystems for processing.

Cross-Asset Risk Analysis

Mapping indicates that multiple assets are concentrated on certain points in the system:

The Sales subsystem is a considerable interest since it contains both payment and order information.

The Admin subsystem is a control point and has multiple assets.

Also, major trust boundaries are:

- Public API endpoints opened by the Storefront
- External payment gateway integrations

These areas have the highest degree of vulnerability to external attacks.

Security Implications

Even though Bagisto has a modular system, its critical assets are still functionally consolidated in a few subsystems. This is a precarious situation because compromising one core module can have widespread ramifications in the system.

There should be strong access control, secure integration, and thorough input validation in the subsystems to control the situation.

Detailed Design

Overview

In this section, I've provided the specific design of the Bagisto system from a classes perspective. The design covers three main areas:

- User Authentication
- Order Processing
- Product Management

The objective is to demonstrate the interaction of key classes and how responsibilities are allocated throughout the system.

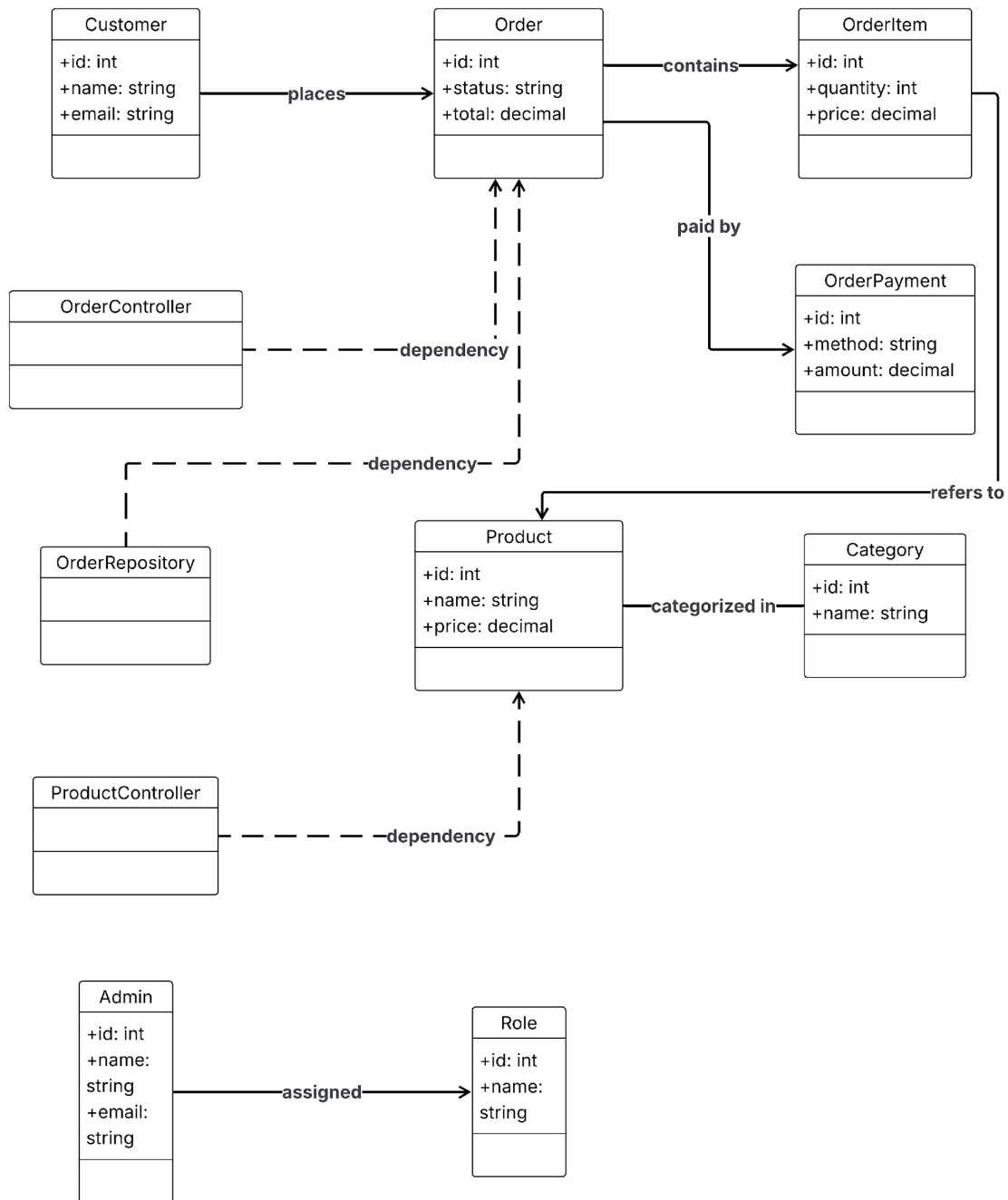


Figure 4 UML Class Diagram of Bagisto System (Derived from Bagisto source code and implemented using Lucidchart)

User Authentication Design

The authentication mechanism is based on the authentication model provided by Laravel, which is enriched through classes specific to Bagisto.

The primary entities are:

- Customer
- Admin
- Role

The Customer and Admin classes build on the authentication model of Laravel and contain the user credentials and profile data. The Admin class is linked to a Role that specifies access rights.

Session controllers handle the logic of authentication, and the Bouncer service implements the logic of authorization.

Relationships:

- An Admin is linked to a Role
- A Customer may have several Orders
- Controllers handle the authentication process through the use of guards

Security Insight:

This design implements role-based access control (RBAC) to ensure that not all administrative actions are allowed to all users.

Order Processing Design

The order processing subsystem is one of the most intricate and important components of the system.

The primary classes are:

- Order
- OrderItem
- OrderPayment

The Order class is the main entity and is responsible for maintaining relationships with order items, order payments, order invoices, order shipments, and order refunds.

The OrderRepository class handles the business logic associated with the creation of an order, the changing of an order status, and the calculation of an order total.

The checkout process is controlled by the OnepageController class, which controls the entire order process.

Relationships:

- A Customer can make multiple Orders
- An Order can have multiple OrderItems
- An Order has a single OrderPayment

Security Insight:

Consolidating business logic in repositories aids in validation enforcement and minimizes the potential for inconsistent or insecure actions.

Product Management Design

This subsystem for product management concerns itself with product data, product categories, and product inventory.

The main classes include:

- Product
- Category

The Product class sustains various relations such as categories, inventory sources, and variants. ProductRepository is tasked with product operations such as creation, updates, and searching.

The ProductController is in charge of administrative tasks, while a different controller is in charge of customer operations in the storefront.

Relations:

- A Product is part of a Category
- A Product can have several Variants
- A Product is associated with Inventory Sources

Security Insight:

The subsystem's dynamic attributes and content will require the strict implementation of input controls to avoid injection and XSS attacks.

Key Relationships Summary

The relationships below summarize the order of the system:

- Customer → Order (one to many)
- Order → OrderItem (one to many)
- Order → OrderPayment (one to one)
- Product → Category (many to many)
- Admin → Role (many to one)

These relationships form the foundational structure of the Bagisto system.

Design Evaluation

The system is designed in a modular and layered pattern.

- The Model-View-Controller (MVC) architecture facilitates concern separation.

- The business logic is centralized with the Repository Pattern.
- There is an improvement in security with Role-Based Access Control (RBAC).

However, the order and Product subsystem have complex relationships. This increases the chances of errors during implementation if proper validation is not done.

Design Patterns and Security Principles

Bagisto employs several software design patterns and security principles that grow system maintainability and scalability while infusing enhanced security.

Model-View-Controller (MVC) Pattern

Bagisto is built on top of the Laravel framework which uses the MVC architecture. The Model is made of several data entities, i.e., Customer, Order, and Product. The View is represented by the Storefront and Admin, and the Controller is the one that handles the requests and communicates between the Model and the View.

Separation of concerns also helps elevate security by compartmentalizing user input from business logic and the data.

Repository Pattern

With OrderRepository and ProductRepository, among others, the system employs the repository pattern. These repositories act as an intermediary between the controller and the database, centralizing data access and business logic.

This helps keep security checks systematic and helps reduce the possibility of inconsistent validations.

Role-Based Access Control (RBAC)

Bagisto employs RBAC when it comes to administrative access. Each Admin is assigned a Role, which uses permission to access various routes and functionalities.

This also employs the least privilege where an Admin's access is limited to a specific scope and is geared towards preventing any unauthorized move or action.

Defense in Depth

The system has many security controls, including the use of a defense-in-depth strategy, which uses several security layers through authentication guards, and middleware, and input validations.

A defense-in-depth strategy also helps the system to cover for several possible security vulnerabilities and helps retain the system functionality from total compromise.

Separation of Concerns

The system consists of multiple subsystems, including Admin, Storefront, API, and Core Modules. Each subsystem has specific responsibilities, which reduces the dependencies between system components.

This approach decreases the system's overall complexity and limits the fallout from security issues in a specific component.

Secure by Design Principles

Bagisto offers security by design principles - no access to protected resources without authentication, enforced user input validation, and access restrictions using middleware and authorization.

This approach to design embeds security into the system's design architecture and does not consider it as a secondary element of the system.

Team Contribution

Member	Contribution
Nadin Alaskar	Architecture analysis and high-level system diagram
Layan Alhabib	Threat modeling and OWASP Threat Dragon diagram
Norah Alfayez	Asset mapping and security analysis
Fay Alsubaie	Detailed design and class diagram



College of Computer & Information Science

Secure Software Development

CYS402

Bagisto Project - Phase 4

Dr. Zainab Masood

Nadin Alaskar 222410587

Layan Alhabib 222410739

Norah Alfayez 222410215

Fay Alsubaie 222410396

Introduction

This phase aims to conduct a thorough security evaluation of the Bagisto e-commerce system at the code level. This will include an analysis of the system's code using SonarCloud, as well as a manual assessment of the significant sections of the system. The intention is to assess system security, evaluate and understand the importance of security vulnerabilities, and offer the best possible changes to eliminate or mitigate security risks.

Each of the functions within the system is being assessed separately, ensuring at least one participant is covering most subsystems such as authentication, session, and payment processing. This is essential as these subsystems deal with sensitive information and the overall integrity of the system. In addition, SonarCloud was used to identify security issues and hot spots that may not be readily apparent during a manual review.

GitHub Links

- **Layan:** <https://github.com/222410739>
- **Norah:** <https://github.com/Norahallfa>
- **Fay:** <https://github.com/222410396-tech>
- **Nadine:** <https://github.com/Nadinfafa>

Manual Code Review

Admin Login Function

File: SessionController.php

Method: store

Which subsystem or module the function belongs to

Subsystem: Admin Authentication

Link:

<https://github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Admin/src/Http/Controllers/User/SessionController.php - L40>

What the function does

This function allows the administrator login. It checks and validates the credentials for the admin guard and account status, and redirects the user to the correct dashboard or allowed part.

Why did you choose this function

This function was chosen because it is the first point of entry into the system. Administrative login will give full access to all the system data like users, orders, and system data. This function can be severe if this function is vulnerable.

Which assets or components are affected

The affected assets for this function are admin account credentials, session information, and all system functions that can be accessed through the admin console. Using this function can compromise the whole application.

Any security risks or concerns

When testing the session controller, several issues were identified. It appears several security concerns were identified.

Security vulnerabilities / bad practices / design issues

This function does not have explicit rate limit provisions, nor do physical rate limits appear to be in place to prevent this function from being susceptible to brute force control cases. The redirect console in use is a previous turn redirect console that does not apply a previous turn redirect, and permission based routed logic is hosted on the configuration; if this is misconfigured, it can lead to logic that can be breached.

Potential issues if errors are introduced

This function contains many errors, and it would be easy for an attacker to gain access to an admin account, leading to a data breach and system manipulation. This function does not have a protective limit to the number of login attempts.

github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Admin/src/Http/Controllers/User/SessionController.php#L11

bagisto / packages / Webkul / Admin / src / Http / Controllers / User / SessionController.php



devansh-webkul Merge remote-tracking branch 'upstream/2.3' and master updates

Code Blame Executable File · 169 lines (135 loc) · 4.66 KB

```

1  <?php
2
3  namespace Webkul\Admin\Http\Controllers\User;
4
5  use Illuminate\Http\RedirectResponse;
6  use Illuminate\Http\Response;
7  use Illuminate\Support\Collection;
8  use Illuminate\View\View;
9  use Webkul\Admin\Http\Controllers\Controller;
10
11 class SessionController extends Controller
12 {
13     /**
14      * Show the form for creating a new resource.
15      *
16      * @return View
17      */
18     public function create()
19     {
20         if (auth()->guard('admin')->check()) {
21             return redirect()->route('admin.dashboard.index');
22         }
23
24         if (strpos(url()->previous(), 'admin') !== false) {
25             $intendedUrl = url()->previous();
26         } else {
27             $intendedUrl = route('admin.dashboard.index');
28         }
29
30         session()->put('url.intended', $intendedUrl);
31
32         return view('admin::users.sessions.create');
33     }
34
35     /**
36      * Store a newly created resource in storage.
37      *
38      * @return Response
39      */
40     public function store()
41     {
42         $this->validate(request(), [
43             'email' => 'required|email',
44             'password' => 'required',
45         ]);
46
47         $remember = request('remember');
```

Figure 1 Admin SessionController file

```

/**
 * Store a newly created resource in storage.
 *
 * @return Response
 */
public function store()
{
    $this->validate(request(), [
        'email' => 'required|email',
        'password' => 'required',
    ]);

    $remember = request('remember');

    if (! auth()->guard('admin')->attempt(request(['email', 'password']), $remember)) {
        session()->flash('error', trans('admin::app.settings.users.login-error'));

        return redirect()->back();
    }

    if (! auth()->guard('admin')->user()->status) {
        session()->flash('warning', trans('admin::app.settings.users.activate-warning'));

        auth()->guard('admin')->logout();

        return redirect()->route('admin.session.create');
    }

    if (! bouncer()->hasPermission('dashboard')) {
        return $this->redirectToFirstAccessibleRoute();
    }

    return redirect()->intended(route('admin.dashboard.index'));
}

```

Figure 2 Admin Login Function

Customer Login Use Case

File: SessionController.php

Method: store

Link:

<https://github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Shop/src/Http/Controllers/Customer/SessionController.php - L34>

Which subsystem or module the function belongs to

Subsystem: Customer Authentication

What the function does

This function handles customer login process on the platform. Upon login, user data is authenticated, account status is verified, email is confirmed, and a session is created. After the session is created, an event is raised to perform cart items initialization.

Why did you choose this function

This function is essential as it handles the login and session driving processes, as it is required for all web applications. The function also touches user confidential and sensitive data.

Which assets or components are affected

The impacted assets are customer login credentials, session cookies, and user account data. As the function is responsible for login session creation, it also controls subsequent user account activities, such as shopping and updating user profile.

Any security risks or concernsSecurity vulnerabilities / bad practices / design issues

The presence of multiple error messages for given failure reasons (e.g., bad credentials, inactive accounts and accounts that remain unverified) raises the risk of account enumeration and unverified user distinguishing. Additionally, session cookies are used temporarily for storing email verification, which raises the risk of client-side data disclosure.

Potential issues if errors are introduced

The lack of explicit rate limiting also increases the impact of a successful search for an account verification bypass. If such verification is successful, it is very likely that actual login credentials would be exploited, the function would fail to capture multiple attempts, thereby inviting customers to fully utilize the system.

github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Shop/src/Http/Controllers/Customer/SessionController.php#L34

bagisto / packages / Webkul / Shop / src / Http / Controllers / Customer / SessionController.php

Code Blame Executable File · 93 lines (70 loc) · 2.55 KB

```

13 class SessionController extends Controller
14 {
15     public function index()
16     {
17     }
18 }
19
20 /**
21  * Show the form for creating a new resource.
22  *
23  * @return Response
24  */
25 public function store(LoginRequest $loginRequest)
26 {
27     $credentials = $loginRequest->only(['email', 'password']);
28
29     $credentials['channel_id'] = core()->getCurrentChannel()->id;
30
31     if (!auth()->guard('customer')->attempt($credentials)) {
32         session()->flash('error', trans('shop::app.customers.login-form.invalid-credentials'));
33
34         return redirect()->back();
35     }
36
37     if (!auth()->guard('customer')->user()->status) {
38         auth()->guard('customer')->logout();
39
40         session()->flash('warning', trans('shop::app.customers.login-form.not-activated'));
41
42         return redirect()->back();
43     }
44
45     if (!auth()->guard('customer')->user()->is_verified) {
46         session()->flash('info', trans('shop::app.customers.login-form.verify-first'));
47
48         Cookie::queue(Cookie::make('enable-resend', 'true', 1));
49
50         Cookie::queue(Cookie::make('email-for-resend', $loginRequest->get('email'), 1));
51
52         auth()->guard('customer')->logout();
53
54         return redirect()->back();
55     }
56 }
57
58 /**
59  * Event passed to prepare cart after login.
60  */
61 Event::dispatch('customer.after.login', auth()->guard()->user());
62
63 if (core()->getConfigData('customer.settings.login_options.redirected_to_page') == 'account') {
64     return redirect()->route('shop.customers.account.profile.index');
65 }
66
67 return redirect()->route('shop.home.index');
68 }
69
70
71
72
73
74
75
76
77

```

Figure 3 Customer SessionController file

```

/**
 * Store a newly created resource in storage.
 *
 * @return Response
 */
public function store()
{
    $this->validate(request(), [
        'email' => 'required|email',
        'password' => 'required',
    ]);

    $remember = request('remember');

    if (! auth()->guard('admin')->attempt(request(['email', 'password']), $remember)) {
        session()->flash('error', trans('admin::app.settings.users.login-error'));

        return redirect()->back();
    }

    if (! auth()->guard('admin')->user()->status) {
        session()->flash('warning', trans('admin::app.settings.users.activate-warning'));

        auth()->guard('admin')->logout();

        return redirect()->route('admin.session.create');
    }

    if (! bouncer()->hasPermission('dashboard')) {
        return $this->redirectToFirstAccessibleRoute();
    }

    return redirect()->intended(route('admin.dashboard.index'));
}

```

Figure 4 Customer Login Function

Cart Merge Function

File: Cart.php

Method: mergeCart

Link:

<https://github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Checkout/src/Cart.php - L208>

Which subsystem or module the function belongs to

Subsystem: Checkout and Session Handling

What the function does

When a guest user logs into their account, their cart becomes merged with their guest cart. Items can then be added to (or removed from) the user cart.

Why did you choose this function

This function performs the most critical segment of this process as it entails handling user sessions and the transfer of data between different states, which can be a particularly sensitive area of operations.

Which assets or components are affected

Integrity of interactive user data (cart data, system data, and user selections) must be preserved as cart data can be particularly indicative of a transaction.

Any security risks or concerns

There is a potential threat associated with the method. The method `mergeCart` relies on session cart identifiers, updating the management of security. If a session ID is highly likely to be targeted, this method can potentially fall prey to a session hijacking. The method also does not handle null checks on cart data, which can result in a runtime error.

Security vulnerabilities / bad practices / design issues

An apparent threat is the method has a blank catch statement where errors are ignored. This is indeed a method in poor security practice as errors should be logged for critical security handling user sessions.

Potential issues if errors are introduced

This method can lead to unauthorized changes to the cart that are due to merging the inconsistencies of data associated with the user on this portal. Subsequently, poor data management can also be presented to the user.

github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Checkout/src/Cart.php#L4

requests 10 Discussions Actions Projects Wiki Security and quality 11 Insights

bagisto / packages / Webkul / Checkout / src / Cart.php

devansh-webkul fix: fixed wishlist quantity issue ✓ 2bb7

Code Blame Executable File · 1204 lines (937 loc) · 37.3 KB

```

1  <?php
2
3  namespace Webkul\Checkout;
4
5  use Illuminate\Support\Facades\Event;
6  use Webkul\Checkout\Contracts\CartAddress as CartAddressContract;
7  use Webkul\Checkout\Exceptions\BillingAddressNotFoundException;
8  use Webkul\Checkout\Models\CartAddress;
9  use Webkul\Checkout\Repositories\CartPayment;
10 use Webkul\Checkout\Repositories\CartAddressRepository;
11 use Webkul\Checkout\Repositories\CartItemRepository;
12 use Webkul\Checkout\Repositories\CartRepository;
13 use Webkul\Customer\Contracts\Customer as CustomerContract;
14 use Webkul\Customer\Contracts\Wishlist as WishlistContract;
15 use Webkul\Customer\Repositories\CustomerAddressRepository;
16 use Webkul\Customer\Repositories\WishlistRepository;
17 use Webkul\Product\Contracts\Product as ProductContract;
18 use Webkul\Product\Repositories\ProductRepository;
19 use Webkul\Shipping\Facades\Shipping;
20 use Webkul\Tax\Facades\Tax;
21 use Webkul\Tax\Repositories\TaxCategoryRepository;
22
23 class Cart
24 {
25     /**
26      * The cart instance.
27      *
28      * @var Contracts\Cart
29      */
30     private $cart;
31
32     /**
33      * Constant for tax calculation based on shipping origin.
34      */
35     const TAX_CALCULATION_BASED_ON_SHIPPING_ORIGIN = 'shipping_origin';
36
37     /**
38      * Constant for tax calculation based on billing address.
39      */
40     const TAX_CALCULATION_BASED_ON_BILLING_ADDRESS = 'billing_address';
41
42     /**
43      *
44      */

```

Figure 5 Cart.php file

```

/**
 * This method handles when guest has some of cart products and then logs in.
 */
public function mergeCart(CustomerContract $customer): void
{
    if (! session()->has('cart')) {
        return;
    }

    $cart = $this->cartRepository->findOneWhere([
        'customer_id' => $customer->id,
        'is_active' => 1,
    ]);

    $guestCart = $this->cartRepository->find(session()->get('cart')->id);

    /**
     * When the logged in customer is not having any of the cart instance previously and are active.
     */
    if (! $cart) {
        $this->cartRepository->update([
            'customer_id' => $customer->id,
            'is_guest' => 0,
            'customer_first_name' => $customer->first_name,
            'customer_last_name' => $customer->last_name,
            'customer_email' => $customer->email,
        ], $guestCart->id);

        session()->forget('cart');

        return;
    }

    $this->setCart($cart);

    foreach ($guestCart->items as $guestCartItem) {
        try {
            $this->addProduct($guestCartItem->product, $guestCartItem->additional);
        } catch (\Exception $e) {
            // Ignore exception
        }
    }

    $this->collectTotals();

    $this->removeCart($guestCart);
}

```

Figure 6 mergeCart Function

PayPal IPN Processing Function

File: Ipn.php

Method: processIpn

Link:

<https://github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Paypal/src/Helpers/Ipn.php> - L46

Which subsystem or module the function belongs to

Subsystem: Payment ProcessingWhat the function does

This function is designed for processing IPN receipt functionality from PayPal. It validates the IPN and retrieves the corresponding order to process further updates to the order status.

Why did you choose this function

This function's involvement with payment processing makes this function highly critical. It affects the confirmation process of payments and the processing of orders.

Which assets or components are affected

Payment assets, orders, and related finite order data records are compromised. The effect of the vulnerability is potential orders that goes against the financial framework, confirms payments loss.

Any security risks or concernsSecurity vulnerabilities / bad practices / design issues

This function may quickly result in the function's heavy reliance on PayPal's IPN data; in addition to validation of IPN the function may do beyond validation, and there is no validation governing replays of the same data as attacks, transaction data is reused maliciously. Further suggests insufficient transaction uniqueness checks, leading potentially to process. If no transaction data, this function handles payment confirmations, financial data may recover.

Potential issues if errors are introduced

This is the function's critical failure condition ensuring confirmation and processing.

github.com/bagisto/bagisto/blob/b91acda6eb4436420667ad4cb0b119045dc6e44a/packages/Webkul/Paypal/src/Helpers/lpn.php#L4

10 Discussions Actions Projects Wiki Security and quality Insights

bagisto / packages / Webkul / Paypal / src / Helpers / lpn.php

Code Blame Executable File · 134 lines (114 loc) · 3.41 KB

```

1  <?php
2
3  namespace Webkul\Paypal\Helpers;
4  ...
5  use Illuminate\Support\Facades\Http;
6  use Webkul\Paypal\Payment\Standard;
7  use Webkul\Sales\Contracts\Order;
8  use Webkul\Sales\Repositories\InvoiceRepository;
9  use Webkul\Sales\Repositories\OrderRepository;
10 use Webkul\Sales\Repositories\OrderTransactionRepository;
11
12 class Ipn
13 {
14     /**
15      * IPI post data.
16      *
17      * @var array
18      */
19     protected $post;
20
21     /**
22      * Order $order
23      *
24      * @var Order
25      */
26     protected $order;
27
28     /**
29      * Create a new helper instance.
30      *
31      * @return void
32      */
33     public function __construct(
34         protected Standard $paypalStandard,
35         protected OrderRepository $orderRepository,
36         protected InvoiceRepository $invoiceRepository,
37         protected OrderTransactionRepository $orderTransactionRepository
38     ) {}
39
40     /**
41      * This function process the IPI sent from paypal end.
42      *

```

Figure 7 PayPal Ipn.php file

```

/**
 * This function process the IPN sent from paypal end.
 *
 * @param array $post
 * @return null|void|\Exception
 */
public function processIpn($post)
{
    $this->post = $post;

    if (! $this->postBack()) {
        return;
    }

    try {
        if (
            isset($this->post['txn_type'])
            && $this->post['txn_type'] !== 'recurring_payment'
        ) {
            $this->order = $this->orderRepository->findOneByField(['cart_id' => $this->post['invoice']]);

            $this->processOrder();
        }
    } catch (\Exception $e) {
        throw $e;
    }
}

```

Figure 8 processIpn Function

Automatic Static Analysis Using SonarCloud

Everyone of us analyzed the code of bagisto in SonarQube Cloud. Here are the results:

Nadin's Overview Page

[Favorite Projects](#) [Assigned Issues](#) [Explore](#)[Up](#)[Nadinfo](#) / [bagisto](#) / [Overview](#)

Overview

[Public](#) • No tags • 322k Lines of Code • Last analysis 11 hours ago

The Overview page has a new look, putting your metrics front and center. [Share your feedback](#)

Project health dashboard

See your project's branch health at a glance by exploring trends and risk breakdowns.

Quality Gate Status



Not computed
Run analysis to generate status

Open Issues

[Overall code](#)

6,719

Duplications

[Overall code](#)

52.1%

— No change (last 30 days)

Coverage

[Overall code](#)

No data available to display

Security snapshot

Security Rating

[Overall code](#)

E

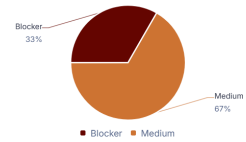
Security Issues

[Overall code](#)

6

— No change (last 30 days)

Open Security Issues by Severity

[Overall code](#)

Security Hotspot snapshot

Fay's Overview Page

Public • No tags • 331k Lines of Code ⓘ • Last analysis 11 minutes ago

Layan's Overview Page

222410739 / bagisto / Overview

Public · No tags · 322k Lines of Code ⓘ · Last analysis 12 hours ago

[View on GitHub](#)

Quality Gate Status

Overall code

—

Not computed
Run analysis to
generate status

Open Issues

Overall code

6,719

— No change (last 30 days)

Duplications

Overall code

52.1%

— No change (last 30 days)

Coverage

Overall code

No data available to display

Security snapshot

Security Rating

Overall code

E

Security Issues

Overall code

6

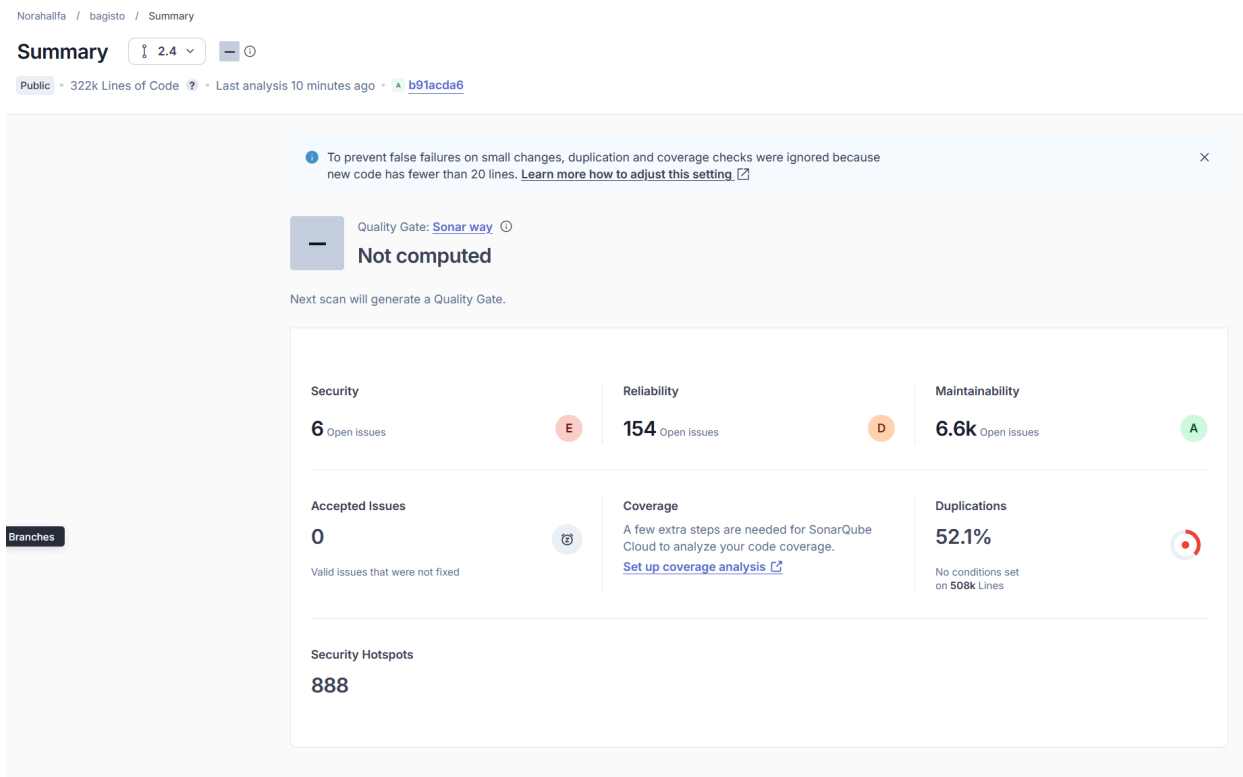
— No change (last 30 days)

Open Security Issues by Severity

Overall code

Severity	Percentage
Blocker	33%
Medium	67%

Norah's Overview Page



Authentication Issue (Nadin)

Link:

https://sonarcloud.io/project/issues?impactSeverities=MEDIUM&impactSoftwareQualities=SECURITY&issueStatuses=OPEN%2CCONFIRMED&id=Norahallfa_bagisto&open=AZ3IxPoYzR5Y5AHZ3d2o

The nature of the problem

The problem relates to the use of the term “password” in the variable name, which can suggest that sensitive data is being processed insecurely. Although this may not always be the case, it can be taken as an indication of hardcoded credentials, and/or insecure use and storage.

Why it is a security risk

Sensitive information, such as passwords, should be handled in a way as to protect the sensitive nature of data. That can lead to unauthorized access and compromise of user accounts.

How to mitigate

Developers should use other storage mechanisms, such as environment variables, rather than hardcoding credentials. Additionally, passwords should be hashed with a strong algorithm and sensitive information should not be logged.

Nadinfo / bagisto / Issues / Detected 'password' in this variable name, review t...

Issues 2.4

2 / 6 issues

config/cors.php

Make sure this permissive CORS policy is safe here.

packages/.../Helpers/Database...

Detected 'password' in this variable name, review this potentially hardcoded credential.

packages/.../RazorpayController...

Change this code to not perform redirects based on user-controlled data.

1 execution flow

packages/.../Feature/LoginPage...

Detected 'password' in this variable name, review this potentially hardcoded credential.

Detected 'password' in this variable name, review this potentially hardcoded credential.

packages/.../AdminFactory.php

Make sure this bcrypt password hash gets revoked, changed, and removed from the code.

Responsibility | Not trustworthy

Detected 'password' in this variable name, review this potentially hardcoded credential.

Credentials should not be hard-coded [php:S2068](#)

Software qualities impacted: Security ● Medium

Open Not assigned Vulnerability Major

Where is the issue? Why is this an issue? How can I fix it? Activity More info

29
30
31
32
33

```
/**
 * Default admin password for which the admin user will be created during installation.
 */
const string DEFAULT_ADMIN_PASSWORD = 'admin123';
```

Detected 'password' in this variable name, review this potentially hardcoded credential.

34 devans..
35
36
37
38
39 anitku..
40 anitku..
41
42
43
44 anitku..
45 anitku..
46 anitku..
47 jltend..
48
49
50
51

```
/**
 * Check if the application is installed.
 */
public function tsInstalled(): bool
{
    if (! file_exists(base_path('.env'))) {
        return false;
    }

    try {
        DB::connection()->getPdo();

        $isConnected = (bool) DB::connection()->getDatabaseName();

        if (! $isConnected) {
            return false;
        }
    }
```

Tags
cwe +

Line affected
L33

Effort
30 min

Introduced
2 months ago

Open in IDE

Regex Denial of Service Hotspot (Layan)

Link:

https://sonarcloud.io/project/security_hotspots?id=222410739_bagisto

The nature of the problem

This hotspot is caused by a regular expression considered to be potentially vulnerable to backtracking. This can cause a large delay in processing inputs of a specific nature.

Why it is a security risk

The main consequence is denial of service, except an attacker can provide backtracking input, which can lead to a large function of processing and can cause the functionality of the system to degrade or cause unavailability of the system.

How to mitigate

To control this, the regular expression should be reduced to a simpler expression, or a safer expression should be provided. Additionally, input length restrictions and execution time limits can reduce the potential that it leads to denial of service.

222410739 / bagisto / Security Hotspots / Make sure the regex used here, which is vulnerabl...

Security Hotspots

2.4

0.0% Security Hotspots Reviewed

To review Fixed Safe

888 Security Hotspots to review

Review priority: Medium

Denial of Service (DoS)

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

Using slow regular expressions is security-sensitive [javascript:S5852](#)

Status: To Review

This Security Hotspot needs to be reviewed to assess whether the code poses a risk.

Review

Where is the risk? What's the risk? Assess the risk How can I fix it? Activity

packages/.../Admin/src/Resources/assets/js/directives/sluggify-target.js

Show 26 lines

Show all lines

```

27      .normalize("NFKD") // Normalize Unicode
28      .replace(/[\u0300-\u036f]/g, "") // Remove combining diacritical marks
29      .replace(/[^a-zA-Z0-9-]/g, "") // Remove all non-letter, non-number characters except spaces and dashes
30      .replace(/\s+/g, "-") // Replace spaces with dashes
31      .replace(/-+/g, "-") // Avoid multiple consecutive dashes
32      .replace(/^+|-+$/g, ""); // Trim leading and trailing dashes

```

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

```

33      if (binding.value) {
34          binding.value(
35

```

Review priority: Medium

Category: Denial of Service (DoS)

Assignee: Not assigned

Open Redirect Issue (Norah)

Link:

https://sonarcloud.io/project/issues?impactSeverities=BLOCKER&issueStatuses=OPEN%2CCONFIRMED&id=Norahallfa_bagisto&open=AZ3IxEaZr5Y5AHZ3d0e

The nature of the problem

This issue refers to the ability to redirect a user based on information that can be, to some extent, controlled by the user. If this is not validated, an attacker can control the redirect destination.

Why it is a security risk

This opens a potential phishing attack vector, where the user is sent to a harmful website that mimics a legitimate service. It also may potentially expose users to additional attacks or data loss.

How to mitigate

The proposed solution is to allow redirects only to known safe routes. It is possible to implement this by using restricted routing or URL validation against a whitelist.

Norahalifa / bagisto / Issues / Change this code to not perform redirects based o...

Issues

2.4

1 / 2 issues

packages/.../RazorpayController...

Change this code to not perform redirects based on user-controlled data.

1 execution flow

- 1 **SOURCE** a user can craft an HTTP request with malicious content
- 2 A malicious value can be assigned to variable '\$errorDescription'
- 3 **SINK** this invocation is not safe; a malicious value can be used as argument

Navigate locations Alt + ▲ ▼

packages/.../AdminFactory.php

Make sure this bcrypt password hash gets revoked, changed, and removed from the code.

2 of 2 shown

Intentionality | Not complete

Change this code to not perform redirects based on user-controlled data. [🔗](#)

HTTP request redirections should not be open to forging attacks [phpsecurity:SS146](#)

Software qualities impacted: Security 🚫 Blocker

☐ Open ☐ Not assigned ☐ Vulnerability ☒ Blocker

Where is the issue? Why is this an issue? How can I fix it? Activity More info

Open in IDE

Tags
cwe +

Line affected
L137

Effort
30 min

Introduced
5 months ago

packages/Webkul/Razorpay/src/Http/Controllers/RazorpayController.php

[See all issues in this file](#)

Show 129 lines

Show all lines

```

130 devans...    /**
131 devans...     * Handle payment error.
132 devans...     */
133 devans...     protected function handlePaymentError(Request $request): RedirectResponse
134 devans...     {
135 devans...         2 $errorDescription = 1 $request->input('error.description', trans('razorpay::app.response.something-went-wrong'));
136 devans...
137 viprny...         3 session()->flash('error', $errorDescription);
138 devans...
139 devans...         return redirect()->route('shop.checkout.cart.index');
140 viprny...     }
141 devans...
142 devans...     /**
143 devans...     * Handle successful payment.
144 devans...     */
145 devans...     protected function handlePaymentSuccess(Request $request, $cart): RedirectResponse
146 devans...     {

```

Show 88 lines

Show all lines

Permissive CORS Policy (Fay)

Link:

https://sonarcloud.io/project/issues?impactSeverities=MEDIUM&impactSoftwareQualities=SECURITY&issueStatuses=OPEN%2CCONFIRMED&id=222410396-tech_bagisto&open=AZ3IwAwDbEa5FZo2nbCK

A permissive CORS policy can expose APIs to unauthorized access, increasing the risk of data leakage or misuse. It may allow malicious websites to interact with the application on behalf of users.

To mitigate this, the application should restrict allowed origins to trusted domains only. It is also advisable to limit allowed methods and headers and to enforce proper authentication mechanisms.

222410396-tech / bagisto / Issues / Make sure this permissive CORS policy is safe here.

Issues 2.4 🔍 🔔

1/4 issues

config/cors.php

Make sure this permissive CORS policy is safe here.

packages/.../Helpers/Database...

Detected 'password' in this variable name, review this potentially hardcoded credential.

packages/.../Feature/LoginPage...

Detected 'password' in this variable name, review this potentially hardcoded credential.

Detected 'password' in this variable name, review this potentially hardcoded credential.

4 of 4 shown

Intentionality | Not complete

Make sure this permissive CORS policy is safe here. 🔗

Cross-Origin Resource Sharing (CORS) policy should be restricted to trusted origins [php:S5122](#)

Software qualities impacted: **Security** 🔴 **Medium**

☐ Open ☐ Not assigned ☐ Vulnerability ☐ Major

Where is the issue? Why is this an issue? How can I fix it? Activity More info

```

15 /
16 */
17
18 'paths' => ['api/*', 'sanctum/csrf-cookie'],
19
20 'allowed_methods' => ['*'],
21
22 'allowed_origins' => ['*'],
23
24 'allowed_origins_patterns' => [],
25
26 'allowed_headers' => ['*'],
27
28 'exposed_headers' => [],
29
30 'max_age' => 0,
31
32 'supports_credentials' => false,
33
34 ];
35

```

Make sure this permissive CORS policy is safe here.

Tags

CWE ... +

Line affected

L22

Effort

4 h

Introduced

4 years ago

Open in IDE

Team Contribution

Member	SonarCloud Issue	Function Reviewed
Nadin Alaskar	Authentication Issue	Admin Login
Layan Alhabib	Regex Hotspot	Customer Login
Norah Alfayez	Open Redirect	Cart Merge
Fay Alsubaie	CORS Issue	PayPal IPN

Conclusion

Both manual and automated methods were utilized in this phase for the security assessment of Bagisto. The manual review allowed for an assessment of risks in critical areas such as authentication, session control, and the payment system. SonarCloud complemented this manual review with insights into risks and potential weaknesses.

The results show that well-defined system architectures may still have security risks when adequate validation, error control, and system parameter setting are neglected. Risk control in the aforementioned areas enhances the security and integrity of the system considerably.

This phase serves to reinforce the idea that security should be woven into the entire system development lifecycle and should not be an incidental function carried out at the system's periphery.